

CSS基础知识

(牛客)为什么要初始化CSS样式?

因为浏览器的兼容问题，不同浏览器对有些标签的默认值是不同的，如果没对CSS初始化往往会出现浏览器之间的页面显示差异。

当然，初始化样式会对SEO有一定的影响，但鱼和熊掌不可兼得，但力求影响最小的情况下初始化。

*最简单的初始化方法就是：`* {padding: 0; margin: 0;}`（不建议）

选择器上的优先级和覆盖原则

对于选择器的**优先级**：

- 标签选择器、伪元素选择器：1
- 类选择器、伪类选择器、属性选择器：10
- id 选择器：100
- 内联样式：1000

注意事项：

- !important 声明的样式的优先级最高；
- 如果优先级相同，则最后出现的样式生效；
- 继承得到的样式的优先级最低；
- 通用选择器（*）、子选择器（>）和相邻同胞选择器（+）并不在这四个等级中，所以它们的权值都为 0；
- 样式表的来源不同时，优先级顺序为：内联样式 > 内部样式 > 外部样式 > 浏览器用户自定义样式 > 浏览器默认样式。

简单记住结论：!important>行内样式>id 选择器>class 选择器/属性选择器>标签选择器>通配符*

覆盖原则：

- 规则一：由于继承而发生样式冲突时，最近祖先获胜。
- 规则二：继承的样式和直接指定的样式冲突时，直接指定的样式获胜。
- 规则三：直接指定的样式发生冲突时，样式权值高者获胜。
- 规则四：样式权值相同时，后者获胜。
- 规则五：!important 的样式属性不被覆盖。

CSS3 中有哪些新特性

- 新增各种 CSS 选择器（: not(.input)：所有 class 不是“input”的节点）
- 圆角（border-radius:8px）
- 多列布局（multi-column layout）
- 阴影和反射（Shadoweflect）
- 文字特效（text-shadow）
- 文字渲染（Text-decoration）
- 线性渐变（gradient）
- 旋转（transform）
- 增加了旋转,缩放,定位,倾斜,动画,多背景

CSS 动画如何实现

创建动画序列，需要使用 `animation` 属性或其子属性，该属性允许配置动画时间、时长 以及其他动画细节，但该属性不能配置动画的实际表现，动画的实际表现是由 `@keyframes` 规则实现。`transition` 也可实现动画。`transition` 强调过渡，是元素的一个或多个属性发生变化时产生的过渡效果，同一个元素通过两个不同的途径获取样式，而第二个途径当某种改变发生（例如 `hover`）时才能获取样式，这样就会产生过渡动画。

animation

值	描述
<code>@keyframes</code>	定义一个动画, <code>@keyframes</code> 定义的动画名称用来被 <code>animation-name</code> 所使用
<code>animation-name</code>	检索或设置对象所应用的动画名称, 必须与规则 <code>@keyframes</code> 配合使用, 因为动画名称由 <code>@keyframes</code> 定义
<code>animation-duration</code>	检索或设置对象动画的持续时间
<code>animation-timing-function</code>	检索或设置对象动画的过渡类型
<code>animation-delay</code>	检索或设置对象动画的延迟时间
<code>animation-iteration-count</code>	检索或设置对象动画的循环次数
<code>animation-direction</code>	检索或设置对象动画在循环中是否反向运动
<code>animation-play-state</code>	检索或设置对象动画的状态

- `animation-timing-function`：检索或设置对象动画的过渡类型

值	描述
<code>linear</code>	动画从头到尾的速度是相同的。
<code>ease</code>	默认。动画以低速开始，然后加快，在结束前变慢。
<code>ease-in</code>	动画以低速开始。
<code>ease-out</code>	动画以低速结束。
<code>ease-in-out</code>	动画以低速开始和结束。
<code>cubic-bezier(n,n,n,n)</code>	在 <code>cubic-bezier</code> 函数中自己的值。可能的值是从 0 到 1 的数值。

- `animation-iteration-count`：检索或设置对象动画的循环次数
 - `infinite`，意思是动画将会无限次的执行，这也就达到了循环的效果，还可以给它具体的数值，当执行你设置的次数后它会自动停止。
- `animation-direction`：检索或设置对象动画在循环中是否反向运动

值	描述
normal	默认值。动画按正常播放。
reverse	动画反向播放。
alternate	动画在奇数次（1、3、5...）正向播放，在偶数次（2、4、6...）反向播放。
alternate-reverse	动画在奇数次（1、3、5...）反向播放，在偶数次（2、4、6...）正向播放。
initial	设置该属性为它的默认值。
inherit	从父元素继承该属性。

transition

值	描述
<i>transition-duration</i>	transition 效果需要指定多少秒或毫秒才能完成
<i>transition-property</i>	指定 CSS 属性的 name，transition 效果
<i>transition-timing-function</i>	指定 transition 效果的转速曲线
<i>transition-delay</i>	定义 transition 效果开始的时候

- transition-property: 指定 CSS 属性的 name，transition 效果

```
div{
  width:100px;
  height:100px;
  border-radius: 50%;
  background:#f40;
  transition-duration:1s;
  transition-property:width;
}
div:hover{
  height:150px;
  width:150px;
}
```

这里transition-property值仅为 width，意思是只给width加动画，所以会呈现这种效果，同样如果换成了height，那么将会是变高才有动画。

CSS 中可继承与不可继承属性有哪些

无继承性的属性

- **display**: 规定元素应该生成的框的类型
- 文本属性
 - vertical-align: 垂直文本对齐
 - text-decoration: 规定添加到文本的装饰
 - text-shadow: 文本阴影效果
 - white-space: 空白符的处理
 - unicode-bidi: 设置文本的方向
- **盒子模型的属性**: width、height、margin、border、padding

- **背景属性**: background、background-color、background-image、background-repeat、background-position、background-attachment
- **定位属性**: float、clear、position、top、right、bottom、left、min-width、min-height、max-width、max-height、overflow、clip、z-index

有继承性的属性

- 字体系列属性
 - font-family: 字体系列
 - font-weight: 字体的粗细
 - font-size: 字体的大小
 - font-style: 字体的风格
- 文本系列属性
 - text-indent: 文本缩进
 - text-align: 文本水平对齐
 - line-height: 行高
 - word-spacing: 单词之间的间距
 - letter-spacing: 中文或者字母之间的间距
 - text-transform: 控制文本大小写（就是 uppercase、lowercase、capitalize 这三个）
 - color: 文本颜色

align-items 和 align-content 的区别

- align-items: 对于每一个单行容器居中，而不是整个。
- align-content: 只适用于多行的容器，并且当交叉轴上有多余的空间，将 flex 线在伸缩容器内对齐。

display的属性值及其作用

属性值	作用
none	元素不显示，并且会从文档流中移除。
block	块类型。默认宽度为父元素宽度，可设置宽高，换行显示。
inline	行内元素类型。默认宽度为内容宽度，不可设置宽高，同行显示。
inline-block	默认宽度为内容宽度，可以设置宽高，同行显示。
list-item	像块类型元素一样显示，并添加样式列表标记。
table	此元素会作为块级表格来显示。
inherit	规定应该从父元素继承display属性的值。

display的block、inline和inline-block的区别

block: 会独占一行，多个元素会另起一行，可以设置width、height、margin和padding属性；

inline: 元素不会独占一行，设置width、height属性无效。但可以设置水平方向的margin和padding属性，不能设置垂直方向的padding和margin；

inline-block: 将对象设置为inline对象，但对象的内容作为block对象呈现，之后的内联对象会被排列在同一行内。对于行内元素和块级元素，其特点如下：

行内元素

- 设置宽高无效;
- 可以设置水平方向的margin和padding属性, 不能设置垂直方向的padding和margin;
- 不会自动换行;

块级元素

- 可以设置宽高;
- 设置margin和padding都有效;
- 可以自动换行;
- 多个块状, 默认排列从上到下。

display: table 和本身的 table 有什么区别

- display:table 和本身 table 是相对应的, 区别在于:
- display:table 的 css 声明能够让一个 html 元素和它的子节点像 table 元素一样, 使用基于表格的 css 布局, 是我们能够轻松定义一个单元格的边界, 背景等样式, 而不会产生因为使用了 table 那样的制表标签导致的语义化问题。之所以现在逐渐淘汰了 table 系表格元素, 是因为用 div+css 编写出来的文件比用 table 边写出来的文件小, 而且 table 必须在页面完全加载后才显示, div 则是逐行显示, table 的嵌套性太多, 没有 div 简洁

隐藏元素的方法有哪些

- **display: none:** 渲染树不会包含该渲染对象, 因此该元素不会在页面中占据位置, 也不会响应绑定的监听事件。
- **visibility: hidden:** 元素在页面中仍占据空间, 但是不会响应绑定的监听事件。
- **opacity: 0:** 将元素的透明度设置为 0, 以此来实现元素的隐藏。元素在页面中仍然占据空间, 并且能够响应元素绑定的监听事件。
- **position: absolute:** 通过使用绝对定位将元素移除可视区域内, 以此来实现元素的隐藏。
- **z-index: 负值:** 来使其他元素遮盖住该元素, 以此来实现隐藏。
- **clip/clip-path:** 使用元素裁剪的方法来实现元素的隐藏, 这种方法下, 元素仍在页面中占据位置, 但是不会响应绑定的监听事件。
- **transform: scale(0,0):** 将元素缩放为 0, 来实现元素的隐藏。这种方法下, 元素仍在页面中占据位置, 但是不会响应绑定的监听事件。

rgba() 和 opacity 的透明效果有什么不同?

rgba()和opacity都能实现透明效果, 但最大的不同是opacity作用于元素, 以及元素内的所有内容的透明度, 而rgba()只作用于元素的颜色或其背景色。(设置rgba透明的元素的子元素不会继承透明效果!)

link和@import的区别

两者都是外部引用CSS的方式, 它们的区别如下:

- link是XHTML标签, 除了加载CSS外, 还可以定义RSS等其他事务; @import属于CSS范畴, 只能加载CSS。
- link引用CSS时, 在页面载入时同时加载; @import需要页面网页完全载入以后加载。
- link是XHTML标签, 无兼容问题; @import是在CSS2.1提出的, 低版本的浏览器不支持。
- link支持使用javascript控制DOM去改变样式; 而@import不支持。

伪类和伪元素

- 伪类就是一个选择处于特定状态的元素的选择器，比如某一个 class 的第一个元素，某个被 hover 的元素等等，我们可以理解成一个特定的 CSS 类，但与普通的类不一样，它只有处于 DOM 树无法描述的状态下才能为元素添加样式，所以将其称为伪类。
- 伪元素和伪类很像，但是伪元素类似于增添一个新的 DOM 节点到 DOM 树中，而不是改变元素的状态。注意了，这里是类似，而不是真的增加一个节点，这也是其被称为伪元素的原因（实质上，元素被创建在文档外）。
- 伪类是操作文档中已有的元素，而伪元素是创建了一个文档外的元素，两者最关键的区分就是这点。此外，为了书写 CSS 时进行区分，一般伪类是单冒号，如: hover，而伪元素是双冒号::before。

盒模型

1.盒模型都是由四个部分组成的，分别是margin、border、padding和content。

2.标准盒模型和IE盒模型的区别在于设置width和height时，所对应的范围不同：

- 标准盒模型的width和height属性的范围只包含了content，
- IE盒模型的width和height属性的范围包含了border、padding和content。

可以通过修改元素的box-sizing属性来改变元素的盒模型：

- box-sizing: content-box表示标准盒模型（默认值）
- box-sizing: border-box表示IE盒模型（怪异盒模型）

CSS预处理器/后处理器是什么？为什么要使用它们？

预处理器，如：less, sass, stylus，用来预编译sass或者less，增加了css代码的复用性。层级，mixin，变量，循环，函数等对编写以及开发UI组件都极为方便。

后处理器，如：postCss，通常是在完成的样式表中根据css规范处理css，让其更加有效。目前最常做的是给css属性添加浏览器私有前缀，实现跨浏览器兼容性的问题。

css预处理器为css增加一些编程特性，无需考虑浏览器的兼容问题，可以在CSS中使用变量，简单的逻辑程序，函数等在编程语言中的一些基本的性能，可以让css更加的简洁，增加适应性以及可读性，可维护性等。

其它css预处理器语言：Sass（Scss），Less, Stylus, Turbine, Switch css, CSS Cacheer, DT Css。

使用原因：

- 结构清晰，便于扩展
- 可以很方便的屏蔽浏览器私有语法的差异
- 可以轻松实现多重继承
- 完美的兼容了CSS代码，可以应用到老项目中

::before 和 :after 的双冒号和单冒号有什么区别？

- 冒号(:)用于CSS2伪类，双冒号(::)用于CSS3伪元素。
- ::before就是以子元素的存在，定义在元素主体内容之前的一个伪元素。并不存在于dom之中，只存在于页面之中。

注意：:before和:after这两个伪元素，是在CSS2.1里新出现的。起初，伪元素的前缀使用的是单冒号语法，但随着Web的进化，在CSS3的规范里，伪元素的语法被修改成使用双冒号，成为::before、::after

双边距重叠问题（外边距重叠）

- 多个相邻（兄弟或者父子关系）普通流的块元素垂直方向 margin 会重叠

- 折叠的结果为：

两个相邻的外边距都是正数时，折叠结果是它们两者之间较大的值。两个相邻的外边距都是负数时，折叠结果是两者绝对值的较大值。两个外边距一正一负时，折叠结果是两者的相加的和。

单行、多行文本溢出隐藏

- 单行文本溢出

```
overflow: hidden;           // 溢出隐藏
text-overflow: ellipsis;    // 溢出用省略号显示
white-space: nowrap;        // 规定段落中的文本不进行换行
```

- 多行文本溢出

```
overflow: hidden;           // 溢出隐藏
text-overflow: ellipsis;    // 溢出用省略号显示
display: -webkit-box;       // 作为弹性伸缩盒子模型显示。
-webkit-box-orient: vertical; // 设置伸缩盒子的子元素排列方式：从上到下垂直排列
-webkit-line-clamp: 3;      // 显示的行数
```

注意：由于上面的三个属性都是 CSS3 的属性，没有浏览器可以兼容，所以要在前面加一个-webkit- 来兼容一部分浏览器。

（牛客）要动态改变层中内容可以使用的方法？

- innerHTML, innerText

（其他）style 标签写在 body 后与 body 前有什么区别？

- 页面加载自上而下 当然是先加载样式。
- 写在 body 标签后由于浏览器以逐行方式对HTML文档进行解析，当解析到写在尾部的样式表（外链或写在 style 标签）会导致浏览器停止之前的渲染，等待加载且解析样式表完成之后重新渲染，在windows的IE下可能会出现 FOUC 现象（即样式失效导致的页面闪烁问题）

页面布局单位及设计(重点)

CSS布局单位

常用的布局单位包括像素（px），百分比（%），em，rem，vw/vh。

（1）像素（px）是页面布局的基础，一个像素表示终端（电脑、手机、平板等）屏幕所能显示的最小区域，像素分为两种类型：CSS像素和物理像素：

- **CSS像素**：为web开发者提供，在CSS中使用的一个抽象单位；
- **物理像素**：只与设备的硬件密度有关，任何设备的物理像素都是固定的。

（2）百分比（%），当浏览器的宽度或者高度发生变化时，通过百分比单位可以使得浏览器中的组件的宽和高随着浏览器的变化而变化，从而实现响应式的效果。一般认为子元素的百分比相对于直接父元素。

（3）em和rem相对于px更具灵活性，它们都是相对长度单位，它们之间的区别：**em相对于父元素，rem相对于根元素。**

- **em**: 文本相对长度单位。相对于当前对象内文本的字体尺寸。如果当前行内文本的字体尺寸未被人为设置, 则相对于浏览器的默认字体尺寸(默认16px)。(相对父元素的字体大小倍数)。
- **rem**: rem是CSS3新增的一个相对单位, 相对于根元素(html元素)的font-size的倍数。**作用**: 利用rem可以实现简单的响应式布局, 可以利用html元素中字体的大小与屏幕间的比值来设置font-size的值, 以此实现当屏幕分辨率变化时让元素也随之变化。

(4) **vw/vh**是与视图窗口有关的单位, vw表示相对于视图窗口的宽度, vh表示相对于视图窗口高度, 除了vw和vh外, 还有vmin和vmax两个相关的单位。

- vw: 相对于视窗的宽度, 视窗宽度是100vw;
- vh: 相对于视窗的高度, 视窗高度是100vh;
- vmin: vw和vh中的较小值;
- vmax: vw和vh中的较大值;

vw/vh 和百分比很类似, 两者的区别:

- 百分比 (%) : 大部分相对于祖先元素, 也有相对于自身的情况比如 (border-radius、translate 等)
- vw/vm: 相对于视窗的尺寸

px、em、rem的区别

- px是固定的像素, 一旦设置了就无法因为适应页面大小而改变。
- em和rem相对于px更具有灵活性, 他们是相对长度单位, 其长度不是固定的, 更适用于响应式布局。
- em是相对于其父元素来设置字体大小, 这样就会存在一个问题, 进行任何元素设置, 都有可能需要知道他父元素的大小。而rem是相对于根元素, 这就意味着, 只需要在根元素确定一个参考值。

flex布局

容器

首先, 实现 flex 布局需要先指定一个容器, 任何一个容器都可以被指定为 flex 布局, 这样容器内部的元素就可以使用 flex 来进行布局。

```
.container {  
  display: flex | inline-flex;    //可以有两种取值  
}
```

需要注意的是: 当时设置 flex 布局之后, 子元素的 float、clear、vertical-align 的属性将会失效。

有下面六种属性可以设置在容器上, 它们分别是:

- 1、flex-direction
 - 2、flex-wrap
 - 3、flex-flow
 - 4、justify-content
 - 5、align-items
 - 6、align-content
- **flex-direction**: 决定主轴的方向(即项目的排列方向)


```
.container {  
  flex-direction: row | row-reverse | column | column-reverse;  
}
```

默认值：row，主轴为水平方向，起点在左端。



https://blog.csdn.net/qq_41799666 牛客@AprilEcho

row-reverse：主轴为水平方向，起点在右端



https://blog.csdn.net/qq_41799666 牛客@AprilEcho

column：主轴为垂直方向，起点在上沿



https://blog.csdn.net/qq_41799666 牛客@AprilEcho

column-reverse：主轴为垂直方向，起点在下沿



牛客@AprilEcho

- **flex-wrap:** 决定容器内项目是否可换行

默认情况下，项目都排在主轴线上，使用 flex-wrap 可实现项目的换行。



牛客@AprilEcho

wrap: 项目主轴总尺寸超出容器时换行，第一行在上方



https://blog.csdn.net/qq_41592261 牛客@AprilEcho

wrap-reverse: 换行，第一行在下方



https://blog.csdn.net/qq_41726604 牛客@AprilEcho

- **flex-flow: flex-direction 和 flex-wrap 的简写形式**

```
.container {  
  flex-flow: <flex-direction> || <flex-wrap>;  
}
```

默认值为: row nowrap(这里是分开的两个属性哦)

- **justify-content: 定义了项目在主轴的对齐方式。**

```
.container {  
  justify-content: flex-start | flex-end | center | space-between | space-around;  
}
```

建立在主轴为水平方向时测试, 即 flex-direction: row

默认值: flex-start 左对齐:



https://blog.csdn.net/qq_41726604

flex-end: 右对齐



牛客@AprilEcho

center: 居中



牛客@AprilEcho

space-between: 两端对齐, 项目之间的间隔相等, 即剩余空间等分成间隙。



space-around: 每个项目两侧的间隔相等，所以项目之间的间隔比项目与边缘的间隔大一倍。



- **align-items:** 定义了项目在交叉轴上的对齐方式

```
.container {
  align-items: flex-start | flex-end | center | baseline | stretch;
}
```

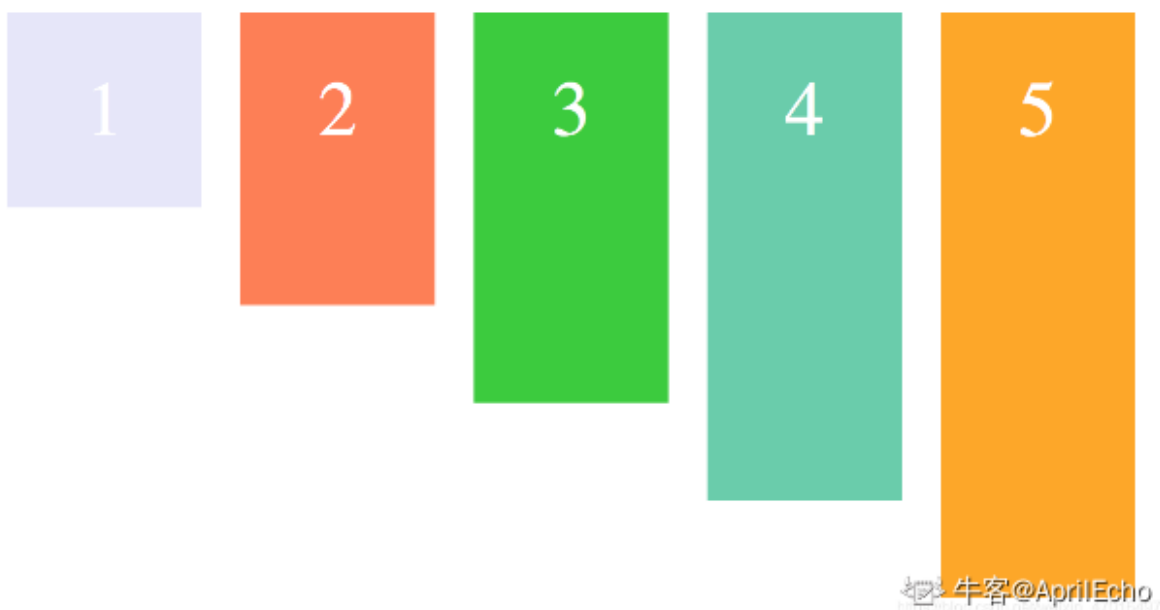
建立在主轴为水平方向时测试，即 flex-direction: row

默认值为 stretch 即如果项目未设置高度或者设为 auto，将占满整个容器的高度。



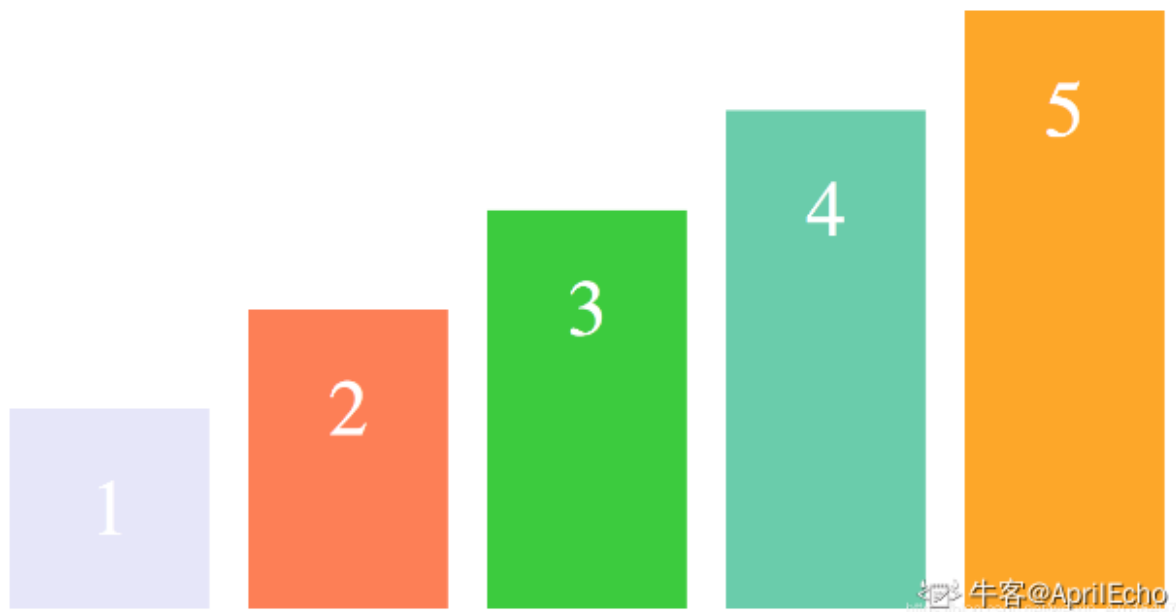
假设容器高度设置为 100px，而项目都没有设置高度的情况下，则项目的高度也为 100px。

flex-start: 交叉轴的起点对齐

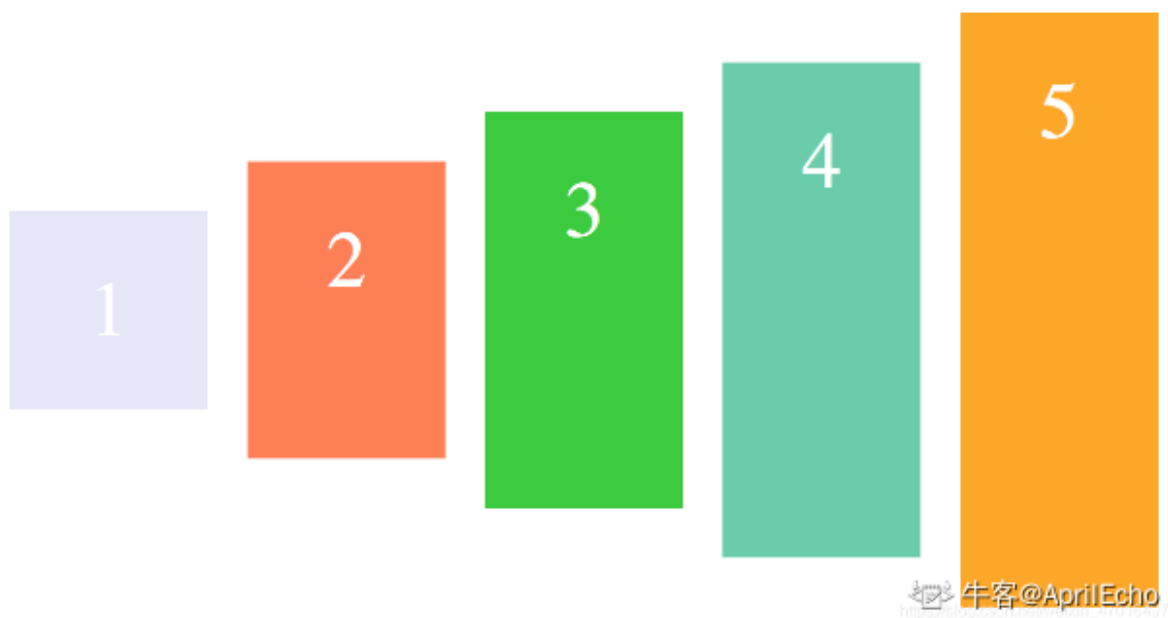


假设容器高度设置为 100px，而项目分别为 20px, 40px, 60px, 80px, 100px，则如上图显示。

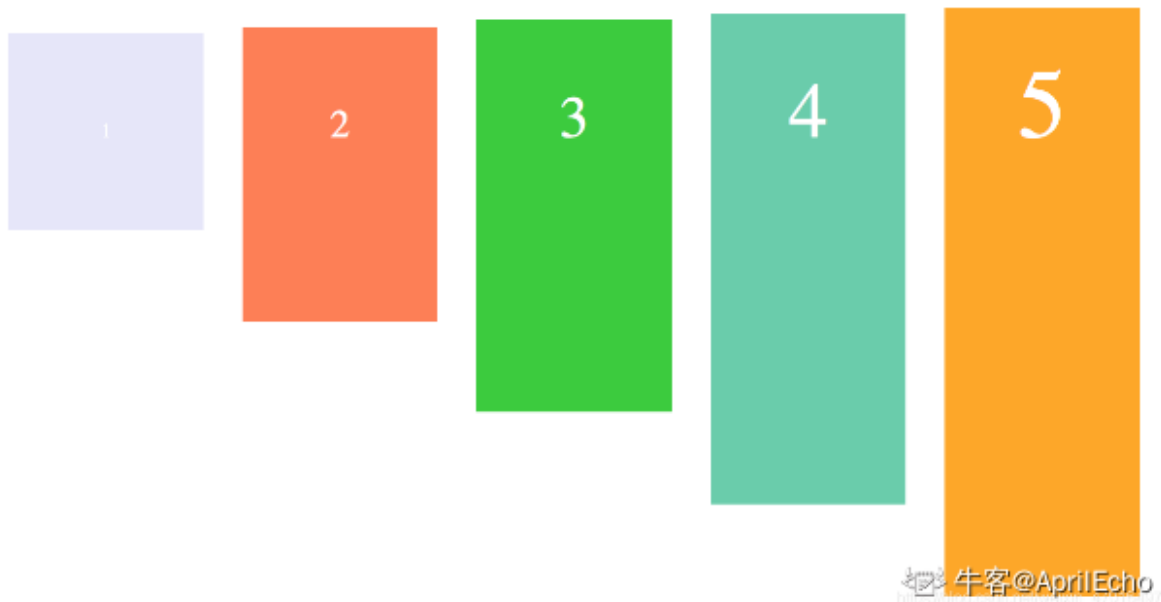
flex-end: 交叉轴的终点对齐



center: 交叉轴的中点对齐



baseline: 项目的第一行文字的基线对齐



- **align-content:** 定义了对多根轴线的对齐方式，如果项目只有一根轴线，那么该属性将不起作用

```
.container {  
  align-content: flex-start | flex-end | center | space-between | space-around  
  | stretch;  
}
```

子容器

上述讲的是一个容器可以设置的属性，下面讲的是容器内的子容器

- flex-grow 属性
 - flex-grow 属性定义子元素或者子容器的放大比例，默认为 0，即如果存在剩余空间，也不放大。



- flex-shrink
 - 属性：
 - flex-shrink 属性定义了项目的缩小比例，默认为 1，即如果空间不足，该项目将缩小。
- flex-basis 属性
 - flex-basis 属性定义了再分配多余空间之前，项目占据的主轴空间（main size）。浏览器根据这个属性，计算主轴是否有多余空间。它的默认值为 auto，即项目的本来大小。

flex简写

flex 简写属性在下面有三个值的定义 默认值为 0 1 auto;

- flex-grow :定义项目的放大比例，默认为 0
- flex-shrink :定义项目的缩小比例,默认为 1
- flex-basis :定义项目在分配多余的空间之前，项目占据的主轴空间 默认为 auto (item 本来大小)

flex:1

完整形式：flex:1 = flex: 1 1 0%;

- flex:1 在父元素尺寸不足的时候，会优先最小化内容尺寸。
- 使用场景：当我们希望元素可以充分的利用剩余的空间，同时不会很多的占用其他同级元素的空间的时候使用。



flex:auto

完整形式：flex:auto = flex: 1 1 auto;

- flex:auto 在父元素尺寸不足的时候，会优先最大化内容尺寸。

- 使用场景：当我们希望元素充分的使用剩余的空间，各自元素按照各自内容进行分配的时候使用。



flex:0

完整形式：flex:0 = flex: 0 1 0%;

- flex:0 :通常表现为 内容最小化宽度
- 使用场景：当希望元素 item 占用最小化的内容宽度的时候

flex:none

完整形式：flex:none = flex:0 0 auto;

- flex:none;表示元素的大小由内容决定，但是 flex-grow, flex-shrink 都是 0，元素没有弹性，通常表现为 内容最大化宽度
- 使用场景：元素的宽度就是内容的宽度，并且内容永远不会换行



常见布局

两栏布局

一般两栏布局指的是左边一栏宽度固定，右边一栏宽度自适应，两栏布局的具体实现：

- 利用浮动，将左边元素宽度设置为 200px，并且设置向左浮动。将右边元素的 margin-left 设置为 200px，宽度设置为 auto（默认为 auto，撑满整个父元素）

```
.outer {
  height: 100px;
}
.left {
  float: left;
  width: 200px;
  background: tomato;
}
.right {
  margin-left: 200px;
  width: auto;
  background: gold;
}
```

- 利用浮动，左侧元素设置固定大小，并左浮动，右侧元素设置 overflow: hidden; 这样右边就触发了 BFC，BFC 的区域不会与浮动元素发生重叠，所以两侧就不会发生重叠。


```
.left{
  width: 100px;
  height: 200px;
  background: red;
  float: left;
}
.right{
  height: 300px;
  background: blue;
  overflow: hidden;
}
```

- 利用 flex 布局，将左边元素设置为固定宽度 200px，将右边的元素设置为 flex:1。

```
.outer {
  display: flex;
  height: 100px;
}
.left {
  width: 200px;
  background: tomato;
}
.right {
  flex: 1;
  background: gold;
}
```

等宽布局

```
<body>
<div id="parent">
  <div class="column">1 <p>我是文字我是文字我输文字我是文字我是文字</p></div>
  <div class="column">2 <p>我是文字我是文字我输文字我是文字我是文字</p></div>
  <div class="column">3 <p>我是文字我是文字我输文字我是文字我是文字</p></div>
  <div class="column">4 <p>我是文字我是文字我输文字我是文字我是文字</p></div>
</div>
</body>
```

- 使用 float 实现

```
#parent {
  margin-left: -20px; /*使整体内容看起来居中,抵消 padding-left 的影响*/
}
.column{
  padding-left: 20px; /*盒子的边距*/
  width: 25%;
  float: left;
  box-sizing: border-box;
  border: 1px solid #000;
  background-clip: content-box; /*背景色从内容开始绘制,方便观察*/
  height: 500px;
}
.column:nth-child(odd){
  background-color: #f00;
}
```

```
.column:nth-child(even){
    background-color: #0f0;
}
```

- 使用 flex 实现

```
#parent {
    margin-left: -15px; /*使内容看起来居中*/
    height: 500px;
    display: flex;
}
.column{
    flex: 1; /*一起平分#parent*/
    margin-left: 15px; /*设置间距*/
}
.column:nth-child(odd){
    background-color: #f00;
}
.column:nth-child(even){
    background-color: #0f0;
}
```

九宫格布局

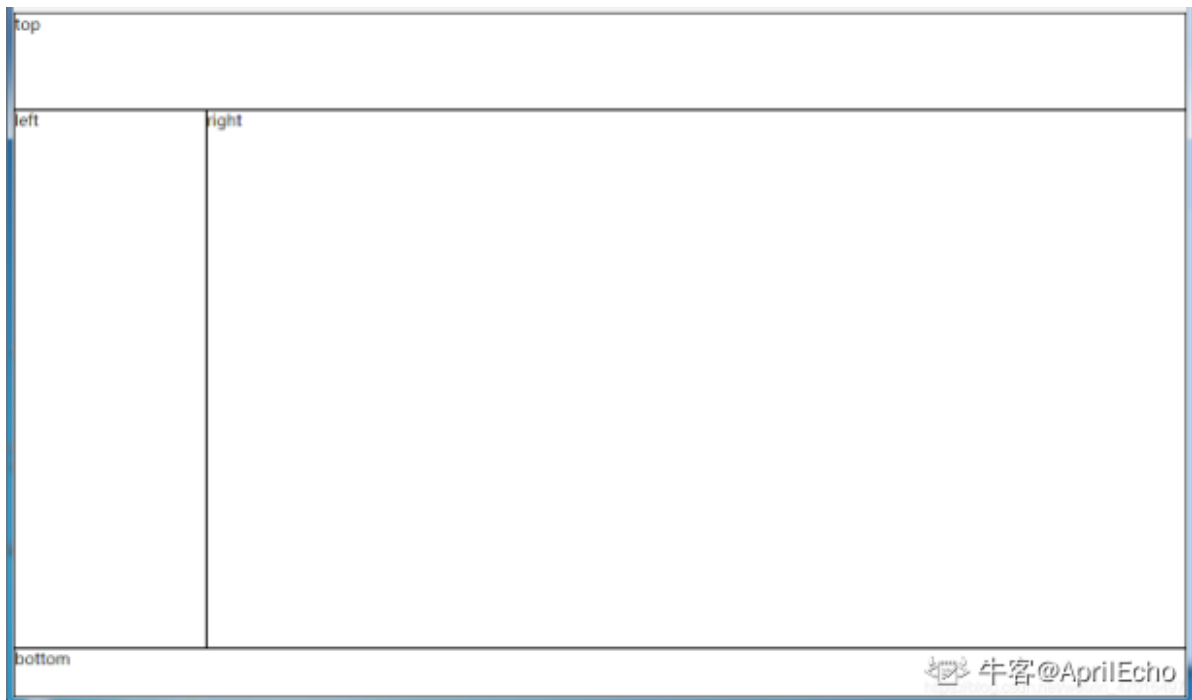
- 使用 flex 实现

```
<body>
<div id="parent">
    <div class="row">
        <div class="item">1</div>
        <div class="item">2</div>
        <div class="item">3</div>
    </div>
    <div class="row">
        <div class="item">4</div>
        <div class="item">5</div>
        <div class="item">6</div>
    </div>
    <div class="row">
        <div class="item">7</div>
        <div class="item">8</div>
        <div class="item">9</div>
    </div>
</div>
</body>
```

```
#parent {
    width: 1200px;
    height: 500px;
    margin: 0 auto;
    display: flex;
    flex-direction: column;
}
.row {
    display: flex;
```

```
    flex: 1;
}
.item {
    flex: 1;
    border: 1px solid #000;
}
```

全屏布局



- 使用绝对定位实现

```
<body>
<div id="parent">
  <div id="top">top</div>
  <div id="left">left</div>
  <div id="right">right</div>
  <div id="bottom">bottom</div>
</div>
</body>
```

```
html, body, #parent {height: 100%;overflow: hidden;}
#parent > div {
  border: 1px solid #000;
}
#top {
  position: absolute;
  top: 0;
  left: 0;
  right: 0;
  height: 100px;
}
#left {
  position: absolute;
```

```

top: 100px; /*值大于等于#top 的高度*/
left: 0;
bottom: 50px; /*值大于等于#bottom 的高度*/
width: 200px;
}
#right {
position: absolute;
overflow: auto;
left: 200px; /*值大于等于#left 的宽度*/
right: 0;
top: 100px; /*值大于等于#top 的高度*/
bottom: 50px; /*值大于等于#bottom 的高度*/
}
#bottom {
position: absolute;
left: 0;
right: 0;
bottom: 0;
height: 50px;
}
}

```

- 使用 flex 实现

```

<body>
<div id="parent">
  <div id="top">top</div>
  <div id="middle">
    <div id="left">left</div>
    <div id="right">right</div>
  </div>
  <div id="bottom">bottom</div>
</div>
</body>

```

```

*{
margin: 0;
padding: 0;
}
html,body,#parent{
height:100%;
}
#parent {
display: flex;
flex-direction: column;
}
#top {
height: 100px;
}
#bottom {
height: 50px;
}
#middle {
flex: 1;
display: flex;
}
#left {

```

```
width: 200px;
}
#right {
  flex: 1;
  overflow: auto;
}
```

圣杯布局

- 圣杯布局是常见的三栏式布局。两边顶宽，中间自适应的三栏布局。

```
<div class="container">  
    <div class="middle">测试测试测试测试测试测试测试测试测试测试测试测试测试测试测试测试测试  
测试测试测试测试测试测试测试测试测试测试测试测试</div>  
    <div class="left">left</div>  
    <div class="right">right</div>  
</div>
```

```
.container{
    padding: 0 200px;
}

.middle{
    width: 100%;
    background: paleturquoise;
    height: 200px;
    float: left;
}

.left{
    background: palevioletred;
    width: 200px;
    height: 200px;
    float: left;
    font-size: 40px;
    color: #fff;
    margin-left: -100% ;
    position: relative;
    left: -200px;
}

.right{
    width: 200px;
    height: 200px;
    background: purple;
    font-size: 40px;
    float: left;
    color: #fff;
    margin-left: -200px;
    position: relative;
    right: -200px;
}
```

双飞翼布局

```
<div class="container">  
    <div class="middle-container">  
        <div class="middle">测试测试测试测试测试测试测试测试测试测试测试测试测试测试测试测试测试测试  
测试测试测试测试测试测试测试测试测试测试测试测试测试测试测试测试测试测试</div>  
    </div>  
    <div class="left">left</div>  
    <div class="right">right</div>  
</div>
```

```
.middle-container{
  width: 100%;
  background: paleturquoise;
  height: 200px;
  float: left;
}

.middle{
  margin-left: 200px;
  margin-right: 200px;
}

.left{
  background: palevioletred;
  width: 200px;
  height: 200px;
  float: left;
  font-size: 40px;
  color: #fff;
  margin-left:-100%;
}

.right{
  width: 200px;
  height: 200px;
  background: purple;
  font-size: 40px;
  float: left;
  color: #fff;
  margin-left:-200px;
}
```

水平垂直居中 (重点)

定宽高

- 绝对定位和负 margin 值

```
<template>
  <div id="app">
    <div class="box">
      <div class="children-box"></div>
    </div>
  </div>
</template>
<style type="text/css">
.box {
```

```

    width: 200px;
    height: 200px;
    border: 1px solid red;
    position: relative;
}
.children-box {
    position: absolute;
    width: 100px;
    height: 100px;
    background: yellow;
    left: 50%;
    top: 50%;
    margin-left: -50px;
    margin-top: -50px;
}
</style>

```

- 绝对定位 + transform

```

<template>
  <div id="app">
    <div class="box">
      <div class="children-box"></div>
    </div>
  </div>
</template>
<style type="text/css">
.box {
  width: 200px;
  height: 200px;
  border: 1px solid red;
  position: relative;
}
.children-box {
  position: absolute;
  width: 100px;
  height: 100px;
  background: yellow;
  left: 50%;
  top: 50%;
  transform: translate(-50%, -50%);
}
</style>

```

- 绝对定位 + left/right/bottom/top + margin

```

<template>
  <div id="app">
    <div class="box">
      <div class="children-box"></div>
    </div>
  </div>
</template>
<style type="text/css">
.box {
  width: 200px;

```

```

    height: 200px;
    border: 1px solid red;
    position: relative;
}
.children-box {
    position: absolute;
    display: inline;
    top: 0;
    left: 0;
    right: 0;
    bottom: 0px;
    background: yellow;
    margin: auto;
    height: 100px;
    width: 100px;
}
</style>

```

- flex 布局

```

<template>
  <div id="app">
    <div class="box">
      <div class="children-box"></div>
    </div>
  </div>
</template>
<style type="text/css">
.box {
  width: 200px;
  height: 200px;
  border: 1px solid red;
  display: flex;
  justify-content: center;
  align-items: center;
}
.children-box {
  background: yellow;
  height: 100px;
  width: 100px;
}
</style>

```

不定宽高

- 绝对定位 + transform

```

<template>
  <div id="app">
    <div class="box">
      <div class="children-box">111111</div>
    </div>
  </div>
</template>
<style type="text/css">
.box {

```



```

    width: 200px;
    height: 200px;
    border: 1px solid red;
    position: relative;
}
.children-box {
    position: absolute;
    background: yellow;
    left: 50%;
    top: 50%;
    transform: translate(-50%, -50%);
}
</style>

```

- table-cell

```

<template>
  <div id="app">
    <div class="box">
      <div class="children-box">111111</div>
    </div>
  </div>
</template>
<style type="text/css">
.box {
  width: 200px;
  height: 200px;
  border: 1px solid red;
  display: table-cell;
  text-align: center;
  vertical-align: middle;
}
.children-box {
  background: yellow;
  display: inline-block;
}
</style>

```

- flex 布局

```

<template>
  <div id="app">
    <div class="box">
      <div class="children-box">11111111</div>
    </div>
  </div>
</template>
<style type="text/css">
.box {
  width: 200px;
  height: 200px;
  border: 1px solid red;
  display: flex;
  justify-content: center;
  align-items: center;
}

```

```
.children-box {  
    background: yellow;  
}  
</style>
```

- flex 变异布局

```
<template>  
    <div id="app">  
        <div class="box">  
            <div class="children-box">1111111</div>  
        </div>  
    </div>  
</template>  
<style type="text/css">  
.box {  
    width: 200px;  
    height: 200px;  
    border: 1px solid red;  
    display: flex;  
}  
.children-box {  
    background: yellow;  
    margin: auto;  
}  
</style>
```

- grid + flex 布局

```
<template>  
    <div id="app">  
        <div class="box">  
            <div class="children-box">1111111</div>  
        </div>  
    </div>  
</template>  
<style type="text/css">  
.box {  
    width: 200px;  
    height: 200px;  
    border: 1px solid red;  
    display: grid;  
}  
.children-box {  
    background: yellow;  
    align-self: center;  
    justify-self: center;  
}  
</style>
```

内联元素居中布局

水平居中

- 行内元素可设置: `text-align: center;`
- flex 布局设置父元素: `display: flex; justify-content: center;`

垂直居中

- 单行文本父元素确认高度: `height === line-height`
- 多行文本父元素确认高度: `display: table-cell; vertical-align: middle;`

块级元素居中布局

水平居中

- 定宽: `margin: 0 auto;`
- 不定宽: 参考上诉例子中不定宽高例子。

垂直居中

- `position: absolute` 设置 `left`、`top`、`margin-left`、`margin-top`(定高);
- `position: fixed` 设置 `margin: auto`(定高);
- `display: table-cell;`
- `transform: translate(x, y);`
- flex(不定高, 不定宽);
- grid(不定高, 不定宽), 兼容性相对较差;

如何居中一个浮动元素

- 设置当前div的宽度, 然后设置`margin-left: 50%; position: relative; left: -250px;`其中的left是宽度的一半。
- 父元素和子元素同时左浮动, 然后父元素相对左移动50%, 再然后子元素相对左移动-50%。
- position定位等等。

box-sizing 语法和基本用处

box-sizing 规定两个并排的带边框的框, 语法为 `box-sizing: content-box/border-box/inherit`

- `content-box`: 宽度和高度分别应用到元素的内容框, 在宽度和高度之外绘制元素的内边距和边框
- `border-box`: 为元素设定的宽度和高度决定了元素的边框盒,
- `inherit`: 继承父元素的 box-sizing

(牛客) 边距折叠的理解

- **外边距折叠**: 相邻的两个或多个外边距 (margin) 在垂直方向会合并成一个外边距 (margin)
相邻: 没有被非空内容、padding、border 或 clear 分隔开的margin特性. 非空内容就是说这元素之间要么是兄弟关系或者父子关系
- **垂直方向外边距合并计算**:
 - a、参加折叠的margin都是正值: 取其中 margin 较大的值为最终 margin 值。
 - b、参与折叠的 margin 都是负值: 取的是其中绝对值较大的, 然后, 从 0 位置, 负向位移。
 - c、参与折叠的 margin 中有正值, 有负值: 先取出负 margin 中绝对值中最大的, 然后, 和正 margin 值中最大的 margin 相加。

定位（重点）

清除浮动

浮动定义

- 非IE浏览器下，容器不设高度且子元素浮动时，容器高度不能被内容撑开。此时，内容会溢出到容器外面而影响布局。这种现象被称为浮动（溢出）。

浮动元素引起的问题

- 父元素的高度无法被撑开，影响与父元素同级的元素
- 与浮动元素同级的非浮动元素会跟随其后
- 若浮动的元素不是第一个元素，则该元素之前的元素也要浮动，否则会影响页面的显示结构

清除浮动的方式

- 给父级div定义 height 属性
- 最后一个浮动元素之后添加一个空的div标签，并添加 clear:both 样式
- 包含浮动元素的父级标签添加 overflow:hidden 或者 overflow:auto
- 使用 :after 伪元素。

```
.clearfix:after{
    content: "\200B";
    display: table;
    height: 0;
    clear: both;
}
.clearfix{
    *zoom: 1;
}
```

BFC

- 块级格式化上下文，是一个独立的渲染区域，并且有一定的布局规则。
 - BFC 区域不会与 float box 重叠
 - BFC 是页面上的一个独立容器，子元素不会影响到外面
 - 计算 BFC 的高度时，浮动元素也会参与计算
- 用于清除浮动，防止 margin 重叠等
- 哪些元素会生成 BFC：
 - 根元素
 - float 不为 none 的元素
 - position 为 fixed 和 absolute 的元素
 - display 为 inline-block、table-cell、table-caption, flex, inline-flex 的元素
 - overflow 不为 visible 的元素

（拓展）BFC、IFC、GFC 和 FFC

BFC

- BFC(Block Formatting Contexts)直译为"块级格式化上下文"。Block Formatting Contexts就是页面上的一个隔离的渲染区域，容器里面的子元素不会在布局上影响到外面的元素，反之也是如此。

- 那BFC一般有什么用呢？比如常见的多栏布局，结合块级元素浮动，里面的元素则是在一个相对隔离的环境里运行。

IFC

- IFC(Inline Formatting Contexts)直译为"内联格式化上下文"，IFC的line box（线框）高度由其包含行内元素中最高的实际高度计算而来（不受到竖直方向的padding/margin影响）
- IFC中的line box一般左右都贴紧整个IFC，但是会因为float元素而扰乱。float元素会位于IFC与line box之间，使得line box宽度缩短。 同个ifc下的多个line box高度会不同。IFC中时不可能有块级元素的，当插入块级元素时（如p中插入div）会产生两个匿名块与div分隔开，即产生两个IFC，每个IFC对外表现为块级元素，与div垂直排列。
- 那么IFC一般有什么用呢？
 - 水平居中：当一个块要在环境中水平居中时，设置其为inline-block则会在外层产生IFC，通过text-align则可以使其水平居中。
 - 垂直居中：创建一个IFC，用其中一个元素撑开父元素的高度，然后设置其vertical-align:middle，其他行内元素则可以在此父元素下垂直居中。

GFC

- GFC(GridLayout Formatting Contexts)直译为"网格布局格式化上下文"，当为一个元素设置display值为grid的时候，此元素将会获得一个独立的渲染区域，我们可以通过在网格容器（grid container）上定义网格定义行（grid definition rows）和网格定义列（grid definition columns）属性各在网格项目（grid item）上定义网格行（grid row）和网格列（grid columns）为每一个网格项目（grid item）定义位置 and 空间。
- 那么GFC有什么用呢，和table又有什么区别呢？首先同样是一个二维的表格，但GridLayout会有更加丰富的属性来控制行列，控制对齐以及更为精细的渲染语义和控制。

FFC

- FFC(Flex Formatting Contexts)直译为"自适应格式化上下文"，display值为flex或者inline-flex的元素将会生成自适应容器（flex container），可惜这个牛逼的属性只有谷歌和火狐支持，不过在移动端也足够了，至少safari和chrome还是OK的，毕竟这两在移动端才是王道。
- Flex Box 由伸缩容器和伸缩项目组成。通过设置元素的 display 属性为 flex 或 inline-flex 可以得到一个伸缩容器。设置为 flex 的容器被渲染为一个块级元素，而设置为 inline-flex 的容器则渲染为一个行内元素。
- 伸缩容器中的每一个子元素都是一个伸缩项目。伸缩项目可以是任意数量的。伸缩容器外和伸缩项目内的一切元素都不受影响。简单地说，Flexbox 定义了伸缩容器内伸缩项目该如何布局。

position的属性

- 这个相信大家都比较熟悉，就简单整理一下概念就好

属性值	概述
absolute	生成绝对定位的元素，相对于static定位以外的一个父元素进行定位。元素的位置通过left、top、right、bottom属性进行规定。
relative	生成相对定位的元素，相对于其原来的位置进行定位。元素的位置通过left、top、right、bottom属性进行规定。
fixed	生成绝对定位的元素，指定元素相对于屏幕视口（viewport）的位置来指定元素位置。元素的位置在屏幕滚动时不会改变，比如回到顶部的按钮一般都是用此定位方式。
static	默认值，没有定位，元素出现在正常的文档流中，会忽略 top, bottom, left, right 或者 z-index 声明，块级元素从上往下纵向排布，行级元素从左向右排列。
inherit	规定从父元素继承position属性的值

position:sticky

用法

- position:sticky 被称为粘性定位元素（stickily positioned element）是计算后位置属性为 sticky 的元素。
- 简单的理解就是：在目标区域以内，它的行为就像 position:relative;在滑动过程中，某个元素距离其父元素的距离达到 sticky 粘性定位的要求时(比如 top: 100px); position:sticky 这时的效果相当于 fixed 定位，固定到适当位置。
- 元素固定的相对偏移是相对于离它最近的具有滚动框的祖先元素，如果祖先元素都不可以滚动，那么是相对于 viewport 来计算元素的偏移量。

使用条件

- 父元素不能 overflow:hidden 或者 overflow:auto 属性。
- 必须指定 top、bottom、left、right 4 个值之一，否则只会处于相对定位
- 父元素的高度不能低于 sticky 元素的高度
- sticky 元素仅在其父元素内生效

offset/scroll/client 各类属性

- clientHeight：表示的是可视区域的高度，不包含 border 和滚动条
- offsetHeight：表示可视区域的高度，包含了 border 和滚动条
- scrollHeight：表示了所有区域的高度，包含了因为滚动被隐藏的部分。
- clientTop：表示边框 border 的厚度，在未指定的情况下一般为 0
- scrollTop：滚动后被隐藏的高度，获取对象相对于由 offsetParent 属性指定的父坐标(css 定位的元素或 body 元素)距离顶端的高度

clientX clientY

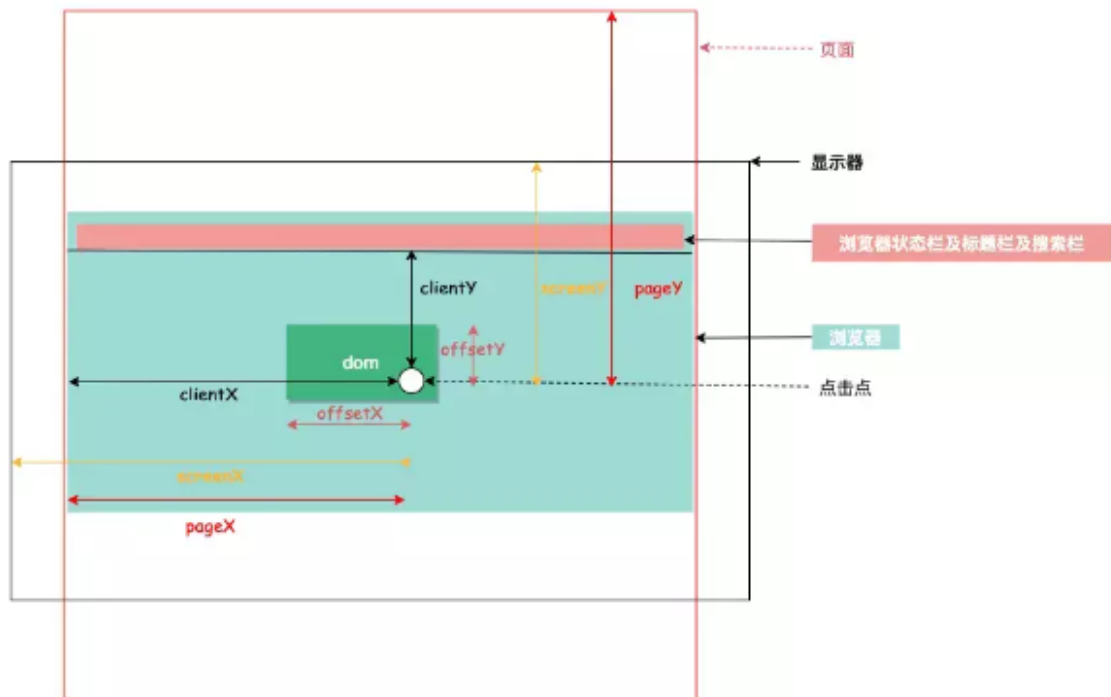
- 鼠标相对于浏览器窗口可视区域的 X, Y 坐标

pageX pageY

- 类似于 clientX, clientY，但它们使用的是文档坐标而非窗口坐标。具体来说, **pageY = clientY + 页面滚动高度**。

offsetX offsetY

- 鼠标相对于事件源元素（srcElement）的 X, Y 坐标。



absolute与fixed共同点与不同点

共同点：

- 改变行内元素的呈现方式，将display置为inline-block
- 使元素脱离普通文档流，不再占据文档物理空间
- 覆盖非定位文档元素

不同点：

- absolute与fixed的根元素不同，absolute的根元素可以设置，fixed根元素是浏览器。
- 在有滚动条的页面中，absolute会跟着父元素进行移动，fixed固定在页面的具体位置。

z-index 的定位方法

z-index 属性设置元素的堆叠顺序，拥有更好堆叠顺序的元素会处于较低顺序元素之前，z-index 可以为负，且 z-index 只能在定位元素上奏效，该属性设置一个定位元素沿 z 轴的位置，如果为正数，离用户越近，为负数，离用户越远，它的属性值有 auto，默认，堆叠顺序与父元素相等，number，inherit，从父元素继承 z-index 属性的值。

z-index属性在什么情况下会失效

通常 z-index 的使用是在有两个重叠的标签，在一定的情况下控制其中一个在另一个的上方或者下方出现。z-index值越大就越是在上层。z-index元素的position属性需要是relative，absolute或是fixed。

z-index属性在下列情况下会失效：

- 父元素position为relative时，子元素的z-index失效。解决：父元素position改为absolute或static；
- 元素没有设置position属性为非static属性。解决：设置该元素的position属性为relative，absolute或是fixed中的一种；
- 元素在设置z-index的同时还设置了float浮动。解决：float去除，改为display: inline-block；

深化

(其他) CSS 优化和提高性能的方法有哪些?

加载性能:

- (1) css压缩: 将写好的css进行打包压缩, 可以减小文件体积。
- (2) css单一样式: 当需要下边距和左边距的时候, 很多时候会选择使用 `margin:top 0 bottom 0; 但 margin-bottom:bottom;margin-left:left;` 执行效率会更高。
- (3) 减少使用@import, 建议使用link, 因为后者在页面加载时一起加载, 前者是等待页面加载完成之后再加载。

选择器性能:

- (1) 关键选择器 (key selector)。选择器的最后面的部分为关键选择器 (即用来匹配目标元素的部分)。CSS选择符是从右到左进行匹配的。当使用后代选择器的时候, 浏览器会遍历所有子元素来确定是否是指定的元素等等;
- (2) 如果规则拥有ID选择器作为其关键选择器, 则不要为规则增加标签。过滤掉无关的规则 (这样样式系统就不会浪费时间去匹配它们了)。
- (3) 避免使用通配规则, 如*{}计算次数惊人, 只对需要用到的元素进行选择。
- (4) 尽量少的去对标签进行选择, 而是用class。
- (5) 尽量少的去使用后代选择器, 降低选择器的权重值。后代选择器的开销是最高的, 尽量将选择器的深度降到最低, 最高不要超过三层, 更多的使用类来关联每一个标签元素。
- (6) 了解哪些属性是可以通过继承而来的, 然后避免对这些属性重复指定规则。

渲染性能:

- (1) 慎重使用高性能属性: 浮动、定位。
- (2) 尽量减少页面重排、重绘。
- (3) 去除空规则: {}。空规则的产生原因一般来说是为了预留样式。去除这些空规则无疑能减少css文档体积。
- (4) 属性值为0时, 不加单位。
- (5) 属性值为浮动小数0.**, 可以省略小数点之前的0。
- (6) 标准化各种浏览器前缀: 带浏览器前缀的在前。标准属性在后。
- (7) 不使用@import前缀, 它会影响css的加载速度。
- (8) 选择器优化嵌套, 尽量避免层级过深。
- (9) css雪碧图, 同一页面相近部分的小图标, 方便使用, 减少页面的请求次数, 但是同时图片本身会变大, 使用时, 优劣考虑清楚, 再使用。
- (10) 正确使用display的属性, 由于display的作用, 某些样式组合会无效, 徒增样式体积的同时也影响解析性能。
- (11) 不滥用web字体。对于中文网站来说WebFonts可能很陌生, 国外却很流行。web fonts通常体积庞大, 而且一些浏览器在下载web fonts时会阻塞页面渲染损伤性能。

可维护性、健壮性:

- (1) 将具有相同属性的样式抽离出来, 整合并通过class在页面中进行使用, 提高css的可维护性。
- (2) 样式与内容分离: 将css代码定义到外部css中。

(其他) CSS预处理器/后处理器

预处理器，如：`less`，`sass`，`stylus`，用来预编译 `sass` 或者 `less`，增加了 `css` 代码的复用性。层级，`mixin`，变量，循环，函数等对编写以及开发UI组件都极为方便。

后处理器，如：`postcss`，通常是在完成的样式表中根据 `css` 规范处理 `css`，让其更加有效。目前最常做的是给 `css` 属性添加浏览器私有前缀，实现跨浏览器兼容性的问题。

`css` 预处理器为 `css` 增加一些编程特性，无需考虑浏览器的兼容问题，可以在 `css` 中使用变量，简单的逻辑程序，函数等在编程语言中的一些基本的性能，可以让 `css` 更加的简洁，增加适应性以及可读性，可维护性等。

其它 `css` 预处理器语言：`Sass (Scss)`，`Less`，`Stylus`，`Turbine`，`Switchch css`，`CSS Cacheer`，`DT CSS`。

使用原因：

- 结构清晰，便于扩展
- 可以很方便的屏蔽浏览器私有语法的差异
- 可以轻松实现多重继承
- 完美的兼容了 `css` 代码，可以应用到老项目中

(其他) Sass、Less 是什么？为什么要使用他们？

他们都是 CSS 预处理器，是 CSS 上的一种抽象层。他们是一种特殊的语法/语言编译成 CSS。例如 Less 是一种动态样式语言，将 CSS 赋予了动态语言的特性，如变量，继承，运算，函数，LESS 既可以在客户端上运行 (支持 IE 6+, Webkit, Firefox)，也可以在服务端运行 (借助 Node.js)。

为什么要使用它们？

- 结构清晰，便于扩展。可以方便地屏蔽浏览器私有语法差异。封装对浏览器语法差异的重复处理，减少无意义的机械劳动。
- 可以轻松实现多重继承。完全兼容 CSS 代码，可以方便地应用到老项目中。LESS 只是在 CSS 语法上做了扩展，所以老的 CSS 代码也可以与 LESS 代码一同编译。

(牛客) 常见浏览器兼容性问题与解决方案？

(1)浏览器兼容问题一：不同浏览器的标签默认的外补丁和内补丁不同

问题症状：随便写几个标签，不加样式控制的情况下，各自的margin 和padding差异较大。

碰到频率:100%

解决方案：CSS里 `*{margin:0;padding:0;}`

这个是最常见的也是最易解决的一个浏览器兼容性问题，几乎所有的CSS文件开头都会用通配符*来设置各个标签的内外补丁是0。

(2)浏览器兼容问题二：块属性标签float后，又有横行的margin情况下，在IE6显示margin比设置的大

问题症状:常见症状是IE6中后面的一块被顶到下一行

碰到频率：90%（稍微复杂点的页面都会碰到，float布局最常见的浏览器兼容问题）

解决方案：在float的标签样式控制中加入 `display:inline;`将其转化为行内属性

我们最常用的就是div+CSS布局了，而div就是一个典型的块属性标签，横向布局的时候我们通常都是用div float实现的，横向的间距设置如果用margin实现，这就是一个必然会碰到的兼容性问题。

(3)浏览器兼容问题三：设置较小高度标签（一般小于10px），在IE6，IE7，遨游中高度超出自己设置高度

问题症状：IE6、7和遨游里这个标签的高度不受控制，超出自己设置的高度

碰到频率：60%

解决方案：给超出高度的标签设置overflow:hidden;或者设置行高line-height 小于你设置的高度。

这种情况一般出现在我们设置小圆角背景的标签里。出现这个问题的原因是IE8之前的浏览器都会给标签一个最小默认的行高的高度。即使你的标签是空的，这个标签的高度还是会达到默认的行高。

(4)浏览器兼容问题四：行内属性标签，设置display:block后采用float布局，又有横行的margin的情况，IE6间距bug

问题症状：IE6里的间距比超过设置的间距

碰到几率：20%

解决方案：在display:block;后面加入display:inline;display:table;

行内属性标签，为了设置宽高，我们需要设置display:block;(除了input标签比较特殊)。在用float布局并有横向的margin后，在IE6下，他就具有了块属性float后的横向margin的bug。不过因为它本身就是行内属性标签，所以我们再加上display:inline的话，它的高宽就不可设了。这时候我们还需要在display:inline后面加入display:table。

(5)浏览器兼容问题五：图片默认有间距

问题症状：几个img标签放在一起的时候，有些浏览器会有默认的间距，加了问题一中提到的通配符也不起作用。

碰到几率：20%

解决方案：使用float属性为img布局

因为img标签是行内属性标签，所以只要不超出容器宽度，img标签都会排在一行里，但是部分浏览器的img标签之间会有个间距。去掉这个间距使用float是正道。（我的一个学生使用负margin，虽然能解决，但负margin本身就是容易引起浏览器兼容问题的用法，所以我禁止他们使用）

(6)浏览器兼容问题六：标签最低高度设置min-height不兼容

问题症状：因为min-height本身就是一个不兼容的CSS属性，所以设置min-height时不能很好的被各个浏览器兼容

碰到几率：5%

解决方案：如果我们要设置一个标签的最小高度200px，需要进行的设置为：{min-height:200px; height:auto !important; height:200px; overflow:visible;}

在B/S系统前端开时，有很多情况下我们又这种需求。当内容小于一个值（如300px）时。容器的高度为300px；当内容高度大于这个值时，容器高度被撑高，而不是出现滚动条。这时候我们就会面临这个兼容性问题。

(7)浏览器兼容问题七：透明度的兼容CSS设置

一般在ie中用的是filter:alpha(opacity=0);这个属性来设置div或者是块级元素的透明度，而在firefox中，一般就是直接使用opacity:0;对于兼容的，一般的做法就是在书写css样式的将2个都写上就行，就能实现兼容

浏览器是怎样解析CSS选择器的？

CSS选择器的解析是从右向左解析的，为了避免对所有元素进行遍历。若从左向右的匹配，发现不符合规则，需要进行回溯，会损失很多性能。若从右向左匹配，先找到所有的最右节点，对于每一个节点，向上寻找其父节点直到找到根元素或满足条件的匹配规则，则结束这个分支的遍历。两种匹配规则的性能差别很大，是因为从右向左的匹配在第一步就筛选掉了大量的不符合条件的最右节点（叶子节点），而从左向右的匹配规则的性能都浪费在了失败的查找上面。

而在CSS解析完毕后，需要将解析的结果与DOM Tree的内容一起进行分析建立一棵Render Tree，最

终用来进行绘图。在建立 Render Tree 时（WebKit 中的「Attachment」过程），浏览器就要为每个 DOM Tree 中的元素根据 CSS 的解析结果（Style Rules）来确定生成怎样的 Render Tree。

响应式设计及其基本原理是什么？

响应式网站设计(Responsive Web design)是一个网站能够兼容多个终端，而不是为每一个终端做一个特定的版本。

基本原理是通过媒体查询检测不同的设备屏幕尺寸做处理。

页面头部必须有meta声明的viewport。

```
<meta name='viewport' content="width=device-width, initial-scale=1\ . maximum-scale=1,user-scalable=no">
```

场景题

实现一个三角形

```
div {  
  width: 0;  
  height: 0;  
  border-bottom: 50px solid red;  
  border-right: 50px solid transparent;  
  border-left: 50px solid transparent;  
}
```

实现一个扇形

```
div{  
  border: 100px solid transparent;  
  width: 0;  
  height: 0;  
  border-radius: 100px;  
  border-top-color: red;  
}
```

画一条0.5px的线

```
transform: scale(0.5,0.5);
```

移动端1px的问题

1px 问题指的是：在一些 Retina屏幕 的机型上，移动端页面的 1px 会变得很粗，呈现出不止 1px 的效果。

解决办法

这个思路就是对 meta 标签里几个关键属性下手：

```
<meta name="viewport" content="initial-scale=0.5, maximum-scale=0.5, minimum-scale=0.5, user-scalable=no">
```

这里针对像素比为2的页面，把整个页面缩放为了原来的1/2大小。这样，本来占用2个物理像素的 1px 样式，现在占用的就是标准的一个物理像素。根据像素比的不同，这个缩放比例可以被计算为不同的值，用 js 代码实现如下：

```
const scale = 1 / window.devicePixelRatio;
// 这里 metaEl 指的是 meta 标签对应的 Dom
metaEl.setAttribute('content', `width=device-width,user-scalable=no,initial-scale=${scale},maximum-scale=${scale},minimum-scale=${scale}`);
```

这样解决了，但这样做的副作用也很大，整个页面被缩放了。这时 1px 已经被处理成物理像素大小，这样的大小在手机上显示边框很合适。但是，一些原本不需要被缩小的内容，比如文字、图片等，也被无差别缩小掉了。

怎么让Chrome支持小于12px 的文字？

```
p{
  font-size:10px;
  -webkit-transform:scale(0.8); //0.8是缩放比例
}
```